

React States and Hooks - Practice Exercises

Questions on React State

1. Create a simple click counter app using React states. You will need a class component called **"MyCounter.jsx"** that will depend on another component called, **"CounterDisplay.jsx"**. **"MyCounter.jsx"** keeps track of the click count value in a state and this is where you will initialize your state that will hold your click counts. It is also under **"MyCounter.jsx"** that you will write a function, called **"allClicksCounter()"** that will update/increase your state by one whenever a button is clicked. Under **"MyCounter.jsx"** you will also need a button which calls the method **"allClicksCounter()"** when it is clicked. Please display the number of clicks right below the button. The main purpose of **"CounterDisplay.jsx"** is to display the number of times the button on the **"MyCounter.js"** component is clicked. Meaning, you will need to pass the updated click data from **"MyCounter.jsx"** to **"CounterDisplay.jsx"** using props.
2. Create another class component named **"EvenCounterDisplay.jsx"**. Please note that **"MyCounter.jsx"** will also depend on **"EvenCounterDisplay.jsx"** to display ONLY the even number of click counts. Meaning, your **"EvenCounterDisplay.jsx"** tracks and displays only the even number of times the button found under **"MyCounter.jsx"** is clicked. Meaning, when the button is clicked just once, your **"MyEvenCounter.jsx"** component should just say **"Clicked 0 times"**. But when the button is clicked again it should say **"Clicked 2 times"**. When the button is clicked for a third time it should just say **"Clicked 2 times"**. When clicked again for the fourth time it should say **"Clicked 4 times"** and so forth. Display the **"MyEvenCounter.jsx"** component right below your **"MyCounter.jsx"** component so that you can see both counters together

****Watch [this demo clip](#) for question 1 & 2, showing what your app should look like****

Question on React Hooks

3. Create a simple click counter app using React Hooks. You will need a functional component called **"IncreaseDecreaseCount.jsx"**. There will be three buttons; a button to increase, a button to decrease and a button to reset click count values and you will need to implement the useState() Hook to update the clicks. Right above the buttons, there will be a count display with an initial click value of 0. When the button to increase is clicked, the click value increases by one, when the decrease button is clicked, the click value will decrease by one and when a

button to reset is clicked, the click value resets to initial value which is 0. Whenever any button is clicked, the change in the click value will be shown on the count displayer.

**** Watch [this demo clip](#) for question 3, showing what your app should look like ****

4. Create a simple theme toggle app using React Hooks. You will need a function component called **“ThemeToggle.jsx”**. In this component, you will use the useState hook to switch between light mode and dark mode. When the app is in light mode, the background color should be #F5F5F5 and text color should be dark and when the app is in dark mode, the background color should be #17212B and text color should be light. The button text should dynamically change based on the current mode (e.g., “Switch to dark mode”/”Switch to light mode”). Note that, the entire component’s background should change based on the selected theme.

****Watch [this demo clip](#) for question 4, showing what your app should look like**

5. Create a simple form using React Hooks by building a functional component called **“PasswordToggle.jsx”**. In this component, you will implement both a username and a password input field. The password input field should include an icon with a clickable feature that allows the user to toggle between showing and hiding the password. When the icon is clicked, the password should switch between hidden (type “password”) and visible (type “text”). You are required to use appropriate icons from React Icons ([Link](#)). The icon must update dynamically based on the current state, meaning when the password is hidden, a “view/show” icon should be displayed, and when the password is visible, a “hide” icon should be displayed. Use the useState() Hook to manage the visibility state of the password and ensure that the toggle functionality works instantly when the icon is clicked, with the icon positioned appropriately inside or next to the input field for a better user experience.

****Watch [this demo clip](#) for question 5, showing what your app should look like**

6. Create a simple click counter app using React Hooks. You will need a functional component called **“UseEffectForTitle.jsx”**. There will be one button and right below the button, there will be a count displayer with an initial click value of 0. When this button is clicked, the click value will increase by one and the displayer will show the changed count value. You will need to implement the useState() Hook to update the click values. When the button is clicked and the click value changes, you will need to change the document’s title to show the count value as well. Basically, the change in the document’s title is a side-effect to the change in the count value due to the button’s click. **Hint:** You will need to use useEffect() to write the

logic/function that will change the document's title to the current value of the click count.

- When the component is rendered for the first time (or when it is mounted), display on the screen an alert text that says “*Component is mounted*”. **Hint:** You will need to use the `useEffect()` Hook to show the alert message and the message should display only when the component is mounted, not every time the button is clicked.

**** Watch [this demo clip](#) for question 6, showing what your app should look like****

***** Steps to follow for each question *****

Steps to follow for question 1

- In your “Evangadi” folder and create a folder named “StatesAndHooksProject”
- Create your React app using vite
- Create Component folder where you will put all your jsx files
- Do not forget to do the following in your "MyCounter.jsx" component:
 - Remember to call your constructor() function in your component
 - Make sure to call the super() method in your constructor
 - Initialize your state that will count the clicks to 0 in the constructor ()
 - Create the “allClicksCounter()” function that will update/increase your state by one. You will need to use the setState() method in your allClicksCounter() function to update your state
 - Render a button which calls the method all “ClicksCounter()” when this button is clicked
 - Pass the updated counter value to the “CounterDisplay.jsx” component to display the number of clicks counted
 - Do not forget to import the "CounterDisplayer.jsx" in your "MyCounter.jsx"
- **Hint:**
 - The "MyCounter.jsx" component is going to be dependent on the "CounterDisplayer.js"
 - The "MyCounter.jsx" component should be the one to be called on the App.js component, not the “CounterDisplayer.js”

- Do not forget to import your **"MyCounter.js"** under **"app.js"**
- You would need to use both states and props for this question

Steps to follow for question 2

- Go to the **"Components"** folder and create a component named **"EvenCounterDisplay.js"**
- Do not forget to do the following in your **"MyCounter.js"** component:
 - Create the **"evenClicksCounter()"** function that will return the click counts that are only even. You will need to use the setState() method in your **"allClicksCounter()"** function to update your state to only contain the even counts
 - Pass the updated counter value to the **"EvenCounterDisplay.js"** component to display the even number of clicks counted
 - Do not forget to import the **"EvenCounterDisplay.js"** in your **"MyCounter.js"**
- **Hint:**
 - The **"MyCounter.js"** component is going to be dependent on the **"MyEvenCounter.js"**
 - Do not forget to import your **"MyEvenCounter.js"** under **"MyCounter.js"**
 - The **"MyCounter.js"** component should be the one to be called on the App.js component, not the **"MyEvenCounter.js"**
 - There will be only one button to click, the button found under **"MyCounter.js"**. When that button is clicked, both **"allClicksCounter()"** function and the **"evenClicksCounter()"** function get executed. Meaning, when the button is clicked, the **"MyCounter.js"** component will show both of the click counters right below the button.
 - **Hint:** You would need to use both states and props for this question

Steps to follow for question 3

- Go to the **"Components"** folder and create a function component called **"IncreaseDecreaseCount.js"**
- Don't forget to do/include these under your **"IncreaseDecreaseCounter.js"** component
 - Importing useState Hook from React
 - Declaring the state variable for the component to hold the increased/decreased click value into your component

- Declaring an updater function that will update the current click count value
- Passing in your initial state value of as an argument to the useState() Hook
- Writing the logic/functions that will increase, decrease and reset your click count values
- Attaching each click event with the respective event handler function
- Import "**IncreaseDecreaseCount.js**" in your App.js and
- Make sure to call the "**IncreaseDecreaseCounter.js**" component in your App.js

Steps to follow for question 4

- Go to your “Components” folder and create a functional component called **ThemeToggle.jsx**
- Declare a state variable like `const [isDarkMode, setIsDarkMode] = useState(false);`.
- Create a toggle function that reverses the current boolean state.
- Apply conditional inline or separate file styling to the component's main `<div>` based on the `isDarkMode` state.
- Dynamically render the button text using a ternary operator (e.g., `isDarkMode ? "Switch to Light Mode" : "Switch to Dark Mode"`)
- Focus mainly on the functionality instead of the UI design.

Step to follow for question 5

- Go to your “Components” folder and create a functional component called **PasswordToggle.jsx**
- Install React Icons if you haven't (`npm install react-icons`).
- Declare a state variable `const [showPassword, setShowPassword] = useState(false);`.
- Create an input field. Set its type attribute dynamically: `type={showPassword ? "text" : "password"}`.
- Render the React Icon next to the input. Use an `onClick` event on the icon to toggle the `showPassword` state.
- Conditionally render the correct icon based on the state.
- Focus mainly on the functionality instead of the UI design.

Steps to follow for question 6

- Go to the “Components” folder and create a function component called **UseEffectForTitle.js**
- Don't forget to do/include these under your **"UseEffectForTitle.js"** component
 - Importing `useState` and `UseEffect` Hooks from `React`
 - Declaring the state variable for the component to hold the click value into your

component

- Declaring an updater function that will update the current click count value
 - Passing in your initial state value of as an argument to the useState() Hook
 - Writing the logic/function that will increase the click count by 1 whenever there is a button click
 - Attaching the click event with the event handler function
 - Writing a function that will alert the “*Component is mounted*” message when the component is mounted (rendered for the first time). You will need to use useEffect() Hook to show this effect
 - Writing a function that will change the document’s title to be the current count value whenever there is a button click. You will need to use useEffect() Hook to show this effect
- Import "UseEffectForTitle.js" in your App.js and
 - Make sure to call the "UseEffectForTitle.js" component in your App.js

Happy coding 😊